

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Programación I

Trabajo Práctico Integrador:

“Gestión de Datos de Países en Python

Filtros, ordenamientos y estadísticas”

Alumno: Troncoso Leonardo Gabriel

Fecha: Junio 2026

Índice

Presentación	3
Objetivo	3
1. Marco Teórico	4
1.1 Listas en Python	4
1.2 Diccionarios en Python	4
1.3 Funciones	5
1.4 Condicionales	6
1.5 Ordenamientos	6
1.6 Estadísticas Básicas	6
1.7 Archivos CSV	7
1.8 Diccionario de datos	7
2. Decisiones Técnicas y Arquitectura	8
2.1 Diagrama de capas	9
2.2 Estructura modular del código	9
2.3 Constantes globales	10
2.4 Módulo csv de Python	10
2.5 Estrategia de escritura	11
2.6 Guardado diferido y confirmación al salir	11
3. Manejo de Errores y Validaciones	11
3.1 Validación al cargar el CSV	11
3.2 Validación de entradas del usuario	12
3.3 Manejo de excepciones de I/O	12
4. Análisis de las Operaciones Principales	12
4.1 Búsqueda	12
4.2 Filtrado	12
4.3 Ordenamiento	13
4.4 Estadísticas	13
5. Capturas de pantalla en tiempo de ejecución	13
6. Dificultades y Conclusiones	15
6.1 Obstáculos técnicos encontrados	15
6.2 Conclusiones grupales	16
7. Links	16
8. Bibliografía / Webgrafía referencias	17

Presentación

El presente informe describe las decisiones técnicas y la arquitectura del programa `gestion_paises.py`, desarrollado como Trabajo Práctico Integrador (TPI) de la materia Programación 1 en el marco de la Tecnicatura Universitaria en Programación a Distancia (TUPaD) de la Universidad Tecnológica Nacional (UTN).

Objetivo:

Desarrollar una aplicación en Python que permita gestionar información sobre países, aplicando listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas. El sistema debe ser capaz de leer datos desde un archivo CSV, realizar consultas y generar indicadores clave a partir del dataset. El objetivo principal es afianzar el uso de estructuras de datos, modularización con funciones y técnicas de filtrado/ordenamiento, aplicando los conceptos aprendidos en Programación 1.

1. Marco Teórico

El sistema implementa un gestor de información geográfica de países que opera completamente desde la terminal, sin dependencias externas más allá de la biblioteca estándar de Python. Permite cargar datos desde un archivo CSV, mantenerlos en memoria durante la sesión, operar sobre ellos a través de un menú interactivo y persistirlos nuevamente en disco al finalizar.

1.1 Listas en Python

Una lista en Python es una estructura de datos ordenada y mutable que puede contener elementos de cualquier tipo de dato. Se define utilizando corchetes `[]` y permite almacenar múltiples valores en una sola variable. Las listas son fundamentales en Python porque ofrecen una manera flexible de agrupar colecciones de datos relacionados.

En este proyecto, la lista principal `países` almacena todos los países del sistema. Cada elemento de esa lista es un diccionario. Se accede a los elementos por índice (`países[0]`), se recorren con `for`, y se filtran generando nuevas listas mediante comprensión de listas. Las funciones `sorted()` y `len()` trabajan directamente sobre listas.

Operaciones principales utilizadas:

- `países.append(nuevo_pais)`: agrega un país al final de la lista.
- `len(países)`: retorna la cantidad de elementos.
- `for pais in países`: recorre cada elemento.
- `sorted(países, key=...)`: devuelve una nueva lista ordenada.
- `[p for p in países if condicion]`: filtra elementos (comprensión de listas).

1.2 Diccionarios en Python

Un diccionario en Python es una colección de pares clave-valor, mutable y no ordenada (aunque desde Python 3.7 mantiene el orden de inserción). Se define con llaves `{}` y permite acceder a valores mediante claves de tipo string, número u otro tipo hashable.

En este sistema, cada país es un diccionario con cuatro claves: `nombre` (str), `poblacion` (int), `superficie` (int) y `continente` (str). Este esquema permite acceder a cualquier dato del país de forma descriptiva (`pais['nombre']`) en lugar de usar índices posicionales, lo que mejora la legibilidad del código.

Ejemplo de diccionario de un país:

```
{  
    "nombre":      "Argentina",  
    "poblacion":   45376763,  
    "superficie":  2780400,  
    "continente":  "América"  
}
```

El diccionario conteo, usado en las estadísticas, acumula la cantidad de países por continente: { 'América': 10, 'Europa': 10, 'Asia': 10, ... }.

1.3 Funciones

Una función en Python es un bloque de código reutilizable que realiza una tarea específica. Se define con la palabra reservada `def`, puede recibir parámetros y retornar valores con `return`. El uso de funciones favorece la modularización: cada función tiene una única responsabilidad, lo que facilita la lectura, el mantenimiento y la detección de errores.

El proyecto está organizado en nueve módulos funcionales, cada uno con sus funciones específicas. Las funciones auxiliares (prefijadas con `_`) son de uso interno y no forman parte de la interfaz pública del módulo. Esto sigue el principio de responsabilidad única (SRP).

Módulos del proyecto:

- Módulo 1: `cargar_paises()` y `guardar_paises()` — lectura/escritura de CSV.
- Módulo 2: `agregar_pais()` y `actualizar_pais()` — alta y modificación.
- Módulo 3: `buscar_pais()` — búsqueda por nombre.
- Módulo 4: `filtrar_paises()` — filtros por continente y rangos numéricos.
- Módulo 5: `ordenar_paises()` — ordenamiento multicriterio.
- Módulo 6: `mostrar_estadisticas()` — indicadores estadísticos.
- Módulo 7: `mostrar_todos()` — visualización completa.
- Módulo 8: funciones auxiliares — validación y formato de entrada/salida.
- Módulo 9: `mostrar_menu()` y `main()` — punto de entrada y bucle principal.

1.4 Condicionales

Las estructuras condicionales (if / elif / else) permiten al programa tomar decisiones en función del valor de una expresión booleana. En Python, los bloques se delimitan únicamente por la indentación, sin usar llaves.

En este proyecto, los condicionales se emplean en múltiples contextos: para validar que los campos no estén vacíos antes de agregar un país, para verificar si un nombre ya existe en la lista (evitar duplicados), para determinar qué campo y dirección de ordenamiento aplicar según la opción elegida, y para controlar el flujo del menú principal.

Ejemplo de uso en la función `_encontrar_extremo()`:

```
for pais in paises[1:]:
    if maximo:
        if pais[campo] > extremo[campo]:
            extremo = pais
    else:
        if pais[campo] < extremo[campo]:
            extremo = pais
```

1.5 Ordenamientos

Python provee la función `sorted()` que devuelve una nueva lista ordenada sin modificar la original. Recibe el parámetro `key` para indicar por qué campo ordenar, y el parámetro `reverse=True` para orden descendente.

El sistema permite ordenar por nombre (string, orden lexicográfico), por población y por superficie (enteros). La función `sorted()` usa internamente el algoritmo Timsort, que garantiza complejidad $O(n \log n)$ en el peor caso.

```
ordenados = sorted(paises, key=lambda p: p[campo],
reverse=descendente)
```

La función `lambda` recibe un diccionario `p` y devuelve el valor del campo indicado, que es lo que `sorted()` usa como criterio de comparación.

1.6 Estadísticas Básicas

Las estadísticas implementadas no requieren librerías externas. Se calculan con funciones propias que recorren la lista una única vez:

- Máximo y mínimo: recorrido lineal con comparación acumulativa ($O(n)$).
- Promedio: suma de todos los valores dividida por la cantidad de países.

- Conteo por continente: diccionario acumulador construido en un único recorrido. Se eligió implementar estas operaciones manualmente (sin usar min(), max() ni sum()) para evidenciar el manejo de bucles e iteraciones propias del nivel de Programación 1.

1.7 Archivos CSV

CSV (Comma-Separated Values) es un formato de texto plano para representar datos tabulares. Cada línea es un registro y los campos se separan por comas. Python incluye el módulo csv en su biblioteca estándar, que provee csv.DictReader (para leer cada fila como diccionario) y csv.DictWriter (para escribir filas desde diccionarios).

La función cargar_paises() abre el archivo con open() en modo lectura, crea un DictReader y valida cada fila antes de agregarla a la lista. Si el archivo no existe, el sistema inicia con lista vacía y lo crea al guardar. Se usa encoding='utf-8' para soportar caracteres especiales como tildes y la letra ñ.

```
with open(ruta, newline='', encoding='utf-8') as archivo:
    lector = csv.DictReader(archivo)
    for fila in lector:
        pais = _validar_fila_csv(fila, numero_fila)
        if pais is not None:
            paises.append(pais)
```

1.8 Diccionario de datos

En la tabla se muestra el diccionario de datos utilizado en el programa.

Entidad: Pais

Campo	Tipo	Unidad	Descripción	Validación
nombre	Texto		Nombre del país	No vacío
Población	Entero	C/u	Cantidad de habitantes	No vacío
Superficie	Entero	Km ²	Kilómetros cuadrados de territorio	No vacío
Continente	Texto		Continente al que pertenece el país	No vacío

2. Decisiones Técnicas y Arquitectura

La arquitectura del sistema sigue el patrón procedimental modular: el programa se divide en funciones independientes, agrupadas temáticamente en módulos lógicos claramente delimitados mediante comentarios de sección. No se emplean clases ni programación orientada a objetos, ya que no fueron tema de estudio en la materia.

El programa aplica de manera consistente la convención de Python de prefijo con guion bajo para funciones de uso interno:

- Funciones sin prefijo (`cargar_paises`, `agregar_pais`, etc.): forman parte de la interfaz pública del módulo, son invocadas directamente desde `main()`.
- Funciones con prefijo `_` (`_validar_fila_csv`, `_buscar_indice`, `_pedir_texto`, etc.): son auxiliares privadas, invocadas únicamente por otras funciones del mismo módulo y no por el menú principal.

Esta distinción mejora la legibilidad del flujo principal: al leer `main()`, el programador puede identificar inmediatamente cuáles son las operaciones de alto nivel sin distraerse con los detalles de implementación.

La función `main()` actúa como punto de entrada y orquestador del sistema. Su lógica sigue tres etapas claramente diferenciadas:

1. Inicialización: invoca `cargar_paises()` y muestra la cantidad de países cargados.
2. Bucle de menú: presenta el menú mediante `mostrar_menu()`, lee la opción del usuario y despacha la función correspondiente utilizando un diccionario de opciones `{"1": mostrar_todos, "2": agregar_pais, ...}`. Este patrón de despacho por diccionario es más extensible y limpio que una cadena de `if/elif`.
3. Terminación: al seleccionar la opción 0, el programa consulta si se desean guardar los cambios antes de finalizar, invocando `guardar_paises()` de ser necesario.

El patrón de guardia al inicio de la mayoría de las funciones operativas (`if not paises: return`) evita errores de ejecución sobre listas vacías y ofrece mensajes de aviso descriptivos al usuario.

El programa utiliza exclusivamente dos módulos de la biblioteca estándar de Python:

- **csv** — para la lectura y escritura del archivo de datos.
- **os** — únicamente para `os.path.exists()`, que verifica si el archivo CSV existe antes de intentar abrirlo.

La decisión de no incorporar bibliotecas de terceros (como `pandas`, `tabulate` o `prettytable`) responde a criterios del enunciado y a la filosofía de la materia: demostrar el dominio de las estructuras nativas de Python (listas, diccionarios, bucles, condicionales) sin delegar en abstracciones de alto nivel.

2.1 Diagrama de capas

Capa	Responsabilidad
Interfaz de usuario	<code>mostrar_menu()</code> + <code>input()</code> — interacción por terminal, lectura de opciones y datos
Lógica de negocio	<code>agregar_pais()</code> , <code>actualizar_pais()</code> , <code>buscar_pais()</code> , <code>filtrar_paises()</code> , <code>ordenar_paises()</code> , <code>mostrar_estadisticas()</code>
Persistencia	<code>cargar_paises()</code> , <code>guardar_paises()</code> — lectura y escritura de archivo CSV
Utilidades	<code>_pedir_texto()</code> , <code>_pedir_entero_positivo()</code> , <code>_convertir_entero_positivo()</code> , <code>_buscar_indice()</code> , <code>_mostrar_lista()</code> , <code>_formato_pais()</code>

2.2 Estructura modular del código

El código fue dividido en nueve módulos bien delimitados por comentarios de sección. Cada función tiene una única responsabilidad. Las funciones auxiliares (prefijadas con guión bajo) encapsulan operaciones repetitivas de validación y formato, evitando duplicar código.

Módulo	Función principal	Responsabilidad
1	<code>cargar_paises()</code> <code>guardar_paises()</code> /	Lectura y escritura del archivo CSV
2	<code>agregar_pais()</code> <code>actualizar_pais()</code> /	Alta de nuevos países y actualización de datos

3	<code>buscar_pais()</code>	Búsqueda parcial por nombre (insensible a mayúsculas)
4	<code>filtrar_paises()</code>	Filtrado por continente, rango de población o superficie
5	<code>ordenar_paises()</code>	Ordenamiento por nombre, población o superficie (asc/desc)
6	<code>mostrar_estadisticas()</code>	Estadísticas: máx, mín, promedios, conteo por continente
7	<code>mostrar_todos()</code>	Visualización tabular de todos los países
8	<code>_pedir_texto()</code> / <code>_pedir_entero_positivo()</code> / etc.	Validación de entradas del usuario
9	<code>main()</code> / <code>mostrar_menu()</code>	Bucle principal y presentación del menú

2.3 Constantes globales

El programa define dos constantes en el nivel de módulo:

- **ARCHIVO_CSV** = "países.csv" — centraliza el nombre del archivo, evitando que esté duplicado en múltiples funciones.
- **CABECERA_CSV** = ["nombre", "poblacion", "superficie", "continente"] — actúa como esquema del CSV; se usa tanto para validar columnas al cargar como para escribir encabezados al guardar.

2.4 Módulo csv de Python

Se utiliza `csv.DictReader` para la lectura y `csv.DictWriter` para la escritura. Esta decisión es coherente con la estructura de datos elegida (lista de diccionarios): `DictReader` produce automáticamente un diccionario por fila usando la primera línea como encabezado, y `DictWriter` escribe filas a partir de diccionarios siguiendo el orden declarado en `fieldnames`. Ambas clases manejan los detalles de escapado y delimitación de manera transparente.

2.5 Estrategia de escritura

`guardar_paises()` sobrescribe el archivo completo en cada llamada, en lugar de aplicar actualizaciones incrementales. Esta estrategia es más simple de implementar y evitar inconsistencias entre el estado en memoria y el estado en disco. La contrapartida es que para archivos muy grandes podría ser ineficiente; sin embargo, dado el alcance del proyecto, es la elección correcta.

2.6 Guardado diferido y confirmación al salir

Los cambios realizados durante la sesión (agregar, actualizar) no se persisten automáticamente: el usuario debe elegir la opción 8 (Guardar) o confirmar el guardado al seleccionar la opción 0 (Salir). Esta decisión permite que el usuario explore modificaciones sin comprometer el archivo, y es análoga al comportamiento de los editores de texto tradicionales.

3. Manejo de Errores y Validaciones

El programa implementa tres niveles de validación diferenciados según el contexto en que ocurren.

3.1 Validación al cargar el CSV

La función `_validar_fila_csv()` actúa como filtro en el momento de la lectura inicial. Por cada fila, verifica:

- Que ningún campo obligatorio esté vacío o compuesto sólo de espacios.
- Que población y superficie sean convertibles a entero (`int()`).
- Que ambos valores sean estrictamente positivos (mayores que cero).

Las filas que no superan alguna de estas comprobaciones se descartan con un mensaje de aviso, pero no interrumpen la carga de las filas restantes. Esto implementa el principio de tolerancia a fallos parciales: un CSV con algunos registros corruptos no impide el funcionamiento del sistema.

3.2 Validación de entradas del usuario

Las funciones auxiliares `_pedir_texto()` y `_pedir_entero_positivo()` encapsulan la lógica de solicitud y validación de datos ingresados por el usuario. `_pedir_entero_positivo()` implementa un bucle de reintento: si el usuario ingresa un valor inválido, el programa muestra un mensaje descriptivo y vuelve a solicitar el dato, hasta que se ingresa un valor correcto o se cancela la operación con la tecla 'q'. Esta decisión mejora la experiencia de uso al no terminar abruptamente ante errores tipográficos.

3.3 Manejo de excepciones de I/O

Las operaciones de apertura y lectura/escritura de archivos están envueltas en bloques `try/except` de excepción genérica (`Exception`). Esto previene que fallos de I/O —como permisos insuficientes, disco lleno o rutas inválidas— propaguen excepciones no controladas al usuario. En su lugar, se muestra un mensaje de error descriptivo y el programa continúa ejecutándose.

4. Análisis de las Operaciones Principales

4.1 Búsqueda

La búsqueda implementada en `buscar_pais()` es de tipo parcial e insensible a mayúsculas: compara el término ingresado contra la versión en minúsculas del nombre de cada país usando el operador `in`. Esta decisión prioriza la usabilidad sobre la exactitud: el usuario no necesita conocer el nombre completo ni respetar las mayúsculas. La complejidad temporal es $O(n)$ sobre la lista completa.

4.2 Filtrado

Los filtros están organizados en un submenú con tres variantes: por continente (coincidencia exacta insensible a mayúsculas), por rango de población y por rango de superficie. Los filtros de rango aceptan mínimo y máximo opcionales: si el mínimo se omite, se asume cero; si el máximo se omite, no hay límite superior. Esto permite filtros unilaterales del estilo «todos los países con más de un millón de habitantes» sin incomodar al usuario con la necesidad de ingresar un valor arbitrariamente grande.

4.3 Ordenamiento

El ordenamiento se realiza con la función built-in `sorted()` de Python, utilizando una función lambda como argumento `key`. Esta decisión es deliberada y didácticamente significativa: `sorted()` implementa el algoritmo Timsort, que es estable y eficiente ($O(n \log n)$ en el caso promedio), y devuelve una nueva lista sin modificar la original, lo que preserva el orden de inserción para operaciones posteriores.

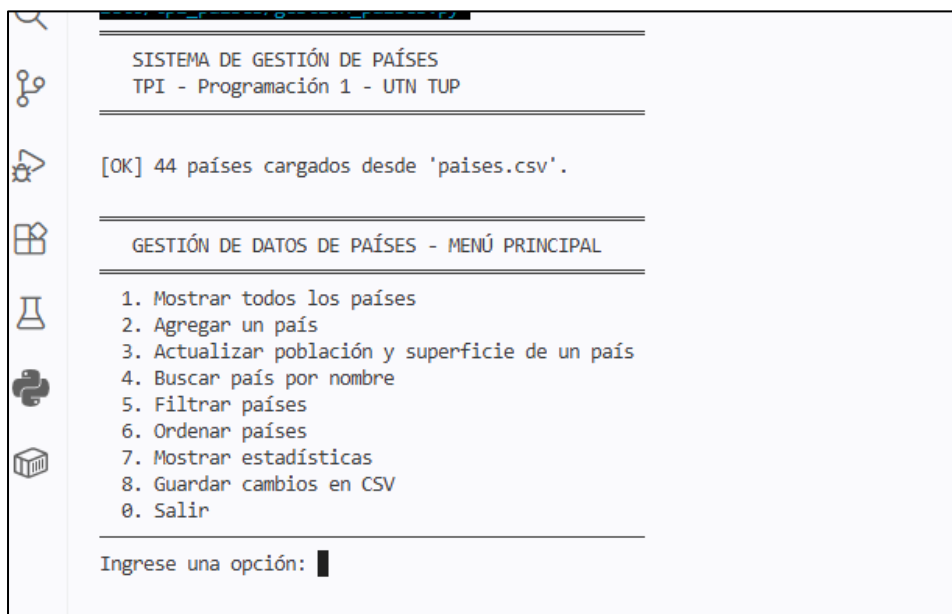
4.4 Estadísticas

Las estadísticas calculadas por `mostrar_estadisticas()` incluyen extremos (país con mayor y menor población), promedios (de población y superficie) y conteo por continente. Cada una de estas operaciones se delega a funciones auxiliares independientes (`_encontrar_extremo()`, `_calcular_promedio()`, `_contar_por_continente()`), lo que facilita su prueba y reutilización aislada. Nótese que `_encontrar_extremo()` implementa el recorrido manual de la lista sin usar las funciones `max()/min()` con `key`, lo cual es una decisión pedagógica explícita del enunciado para mostrar el proceso de comparación elemento a elemento.

5. Capturas de pantalla en tiempo de ejecución

Carga inicial y menú principal

Al iniciar el programa, se carga el CSV y se presenta el menú:



Búsqueda parcial por nombre

Resultado al buscar 'arg':

GESTIÓN DE DATOS DE PAÍSES - MENÚ PRINCIPAL

1. Mostrar todos los países

2. Agregar un país

3. Actualizar población y superficie de un país

4. Buscar país por nombre

5. Filtrar países

6. Ordenar países

7. Mostrar estadísticas

8. Guardar cambios en CSV

0. Salir

Ingrese una opción: 4

— BUSCAR PAÍS —

Ingrese el nombre o parte del nombre a buscar: arg

Se encontraron 2 resultado(s):

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América
Argelia	44,616,624	2,381,741	África

Presione Enter para continuar...

Estadísticas del sistema

Resultado al seleccionar opción 7:

```
GESTIÓN DE DATOS DE PAÍSES - MENÚ PRINCIPAL

1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
0. Salir

Ingrese una opción: 7

— ESTADÍSTICAS —

Total de países cargados : 44

Población:
Mayor → China (1,412,600,000 hab.)
Menor → Fiyi (930,748 hab.)
Promedio → 140,113,748 hab.

Superficie:
Promedio → 1,832,660 km²

Países por continente:
América          10 país/es
Asia              10 país/es
Europa            10 país/es
Oceanía           4 país/es
África            10 país/es

Presione Enter para continuar...
b o A o
```

6. Dificultades y Conclusiones

6.1 Obstáculos técnicos encontrados

Durante el desarrollo del TPI surgieron algunos desafíos que requirieron investigación y adaptación de los conceptos aprendidos en clase:

- Manejo de encoding en CSV: Al cargar el archivo, los caracteres con tilde y la ñ generaban errores en algunos sistemas. La solución fue especificar `encoding='utf-8'` en la función `open()` tanto en la lectura como en la escritura.
- Validación robusta de entradas: Fue necesario diseñar funciones de validación reutilizables (`_pedir_entero_positivo`, `_pedir_texto`) para no repetir el mismo bloque `try/except` en cada función del menú. Esto llevó a comprender mejor el principio de no repetir código (DRY).
- Búsqueda insensible a mayúsculas: La comparación directa de strings fallaba cuando el usuario escribía 'argentina' en lugar de 'Argentina'. Se aplicó el método `.lower()` en ambos lados de la comparación para solucionarlo.
- Evitar modificar la lista durante el ordenamiento: Al usar `sorted()` en vez de `.sort()`, se preserva la lista original y sólo se muestra el resultado ordenado, sin alterar el orden de almacenamiento.
- Filtros de rango con valores opcionales: El filtro por rango necesitaba aceptar que el usuario dejara vacío el mínimo (asumir 0) o el máximo (sin límite superior). Se resolvió con una función auxiliar que retorna `None` cuando el campo está vacío.

6.2 Conclusiones grupales

La realización de este Trabajo Práctico Integrador permitió consolidar y articular los conceptos trabajados a lo largo de la cursada de Programación 1. Más allá de la sintaxis de Python, el desafío principal fue pensar el problema de forma estructurada: descomponerlo en módulos con responsabilidades claras, anticipar casos de error, y diseñar una interfaz de usuario que sea intuitiva y robusta.

Trabajar con archivos CSV fue especialmente valioso porque introdujo la dimensión de la persistencia: los datos ya no viven sólo en la ejecución del programa, sino que pueden guardarse y recuperarse. Esto hizo visible la diferencia entre datos en memoria (la lista `países`) y datos en disco (el archivo CSV).

En síntesis, este proyecto demostró que los conceptos de listas, diccionarios, funciones, condicionales y manejo de archivos no son herramientas aisladas, sino piezas que se combinan para construir sistemas reales y útiles.

7. Links

Link al repositorio del proyecto

Link del video explicativo

8. Bibliografía / Webgrafía referencias

- Van Rossum, G. et al. (2024). Python 3 Documentation – csv module. Python Software Foundation. <https://docs.python.org/3/library/csv.html>
- Van Rossum, G. et al. (2024). Python 3 Documentation – Built-in Functions: sorted(). Python Software Foundation. <https://docs.python.org/3/library/functions.html#sorted>
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- UTN TUPaD (2026). Enunciado del Trabajo Práctico Integrador – Programación 1.
- Real Python. (2024). Dictionaries in Python. <https://realpython.com/python-dicts/>
- UTN TUPaD (2026). Material de clases